UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Object-oriented modeling – III

Readings: ( :chapt. 12)

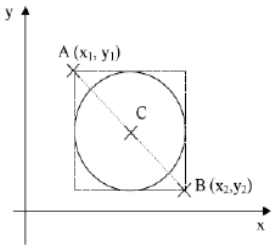
1

Looking for class *candidates*

The task is to develop a *graphic editor* which is able to maintan and operate 2D *geometric figures*. More specifically, the editor should be able of, given a *square*, generate the tangential inner *circumference*.

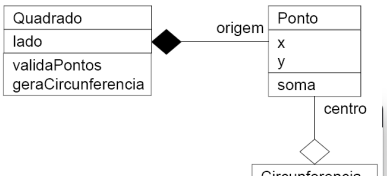
The *square* will be given by 2 suitable *points* defining its *diagonals*. For the sake of simplicity, the *sides* of the *square* are always parallel to the *axis*. Under these conditions, 2 *points* define a *square* when the distance between *coordinates* in x are equal to the distance *coordinates* in y.

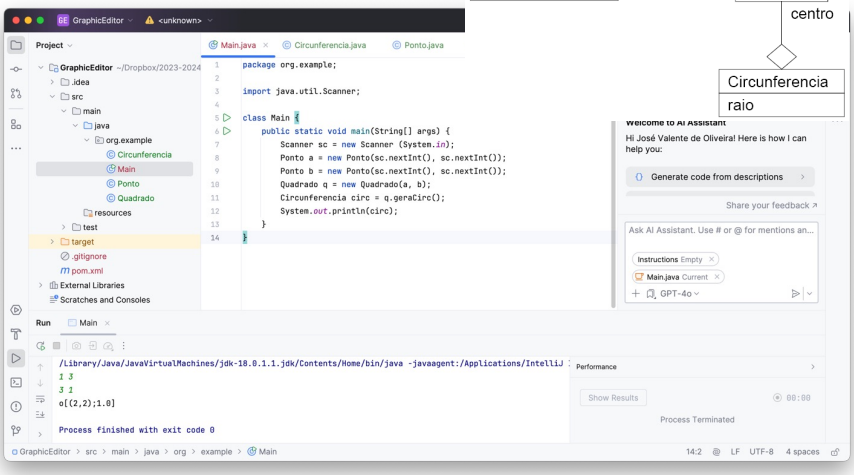
A *circumference* is represented by its *center* (a *point*) and a *radius* (a *float*)



2

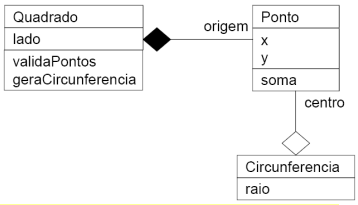
A possible client





3

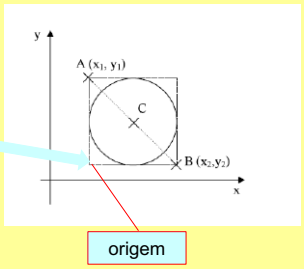
class Quadrado - Constructor




```
public class Quadrado {
private Ponto origem;
private int lado;

public Quadrado (Ponto A, Ponto B) {
    if (validaPontos(A, B)== false) {
            System.err.println("Erro: os pontos "+ A + " e " + B + " nao definem um quadrado");
            System.exit(0);
    }
    origem = A.min(B);
    lado = Math.abs(A.getX()-B.getX());
}

public boolean static validaPontos(Ponto A, Ponto B) {
    int lado1 = Math.abs(A.getX()-B.getX());
    int lado2 = Math.abs(A.getY()-B.getY());
    return lado1 == lado2;
}
```



4

Refining class Ponto

```
Ponto a = new Ponto (1, 2);
Ponto b = new Ponto (2, 0);
Ponto c = a.soma(b);
System.out.println(c); // (3, 2)
```

```
class Ponto {
// …

public Ponto soma(Ponto that) {
   int x = this.getX() + that.getX();
   int y = this.getY() + that.getY();
   return new Ponto(x, y);
}

public Ponto min(Ponto that) {
   int x = Math.min(this.getX(), that.getX());
   int y = Math.min(getY(), that.getY());
   return new Ponto(x, y);
}

public String toString() {
   return "("+getX() + "," + getY()+")";
   }
}
```

5

Class Quadrado – Conclusion

```
public class Quadrado {
// …

public Circunferencia geraCircunferencia() {
     float raio = lado / 2.0f;
     Ponto tmp = new Ponto((int) raio, (int) raio);
     Ponto centro = origem.soma(tmp);
     Circunferencia c = new Circunferencia (centro, raio);
     return c;
}

}
```

6

Refining class Ponto

```
class Ponto {
// …

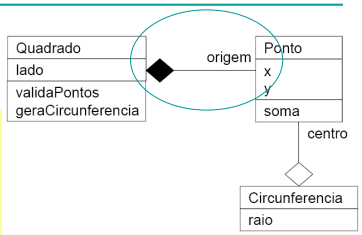
public Ponto soma(Ponto that) {
    int x = this.getX() + that.getX();
    int y = this.getY() + that.getY();
    return new Ponto(x, y);
}

public Ponto min(Ponto that) {
    int x = Math.min(this.getX(), that.getX());
    int y = Math.min(getY(), that.getY());
    return new Ponto(x, y);
}

public String toString() {
    return "("+getX() + "," + getY()+")";
    }
}
```

7

class Quadrado - Constructor



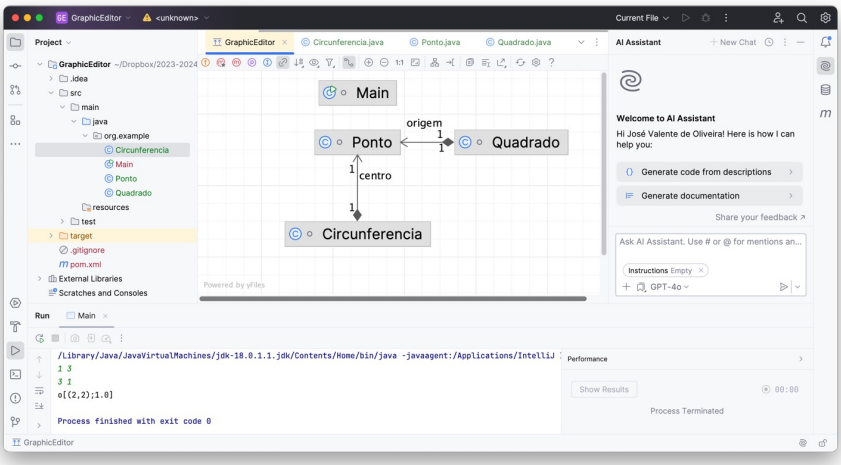

```
public class Quadrado {
private Ponto origem;
private int lado;

public Quadrado (Ponto A, Ponto B) {
      if (validaPontos(A, B)== false) {
            System.err.println("Erro: os pontos "+ A + " e " + B + " nao definem um quadrado");
            System.exit(0);
      }
      origem = A.min(B);
      lado = Math.abs(A.getX()-B.getX());
}

public boolean static validaPontos(Ponto A, Ponto B) {
      int lado1 = Math.abs(A.getX()-B.getX());
      int lado2 = Math.abs(A.getY()-B.getY());
      return lado1 == lado2;
}
```

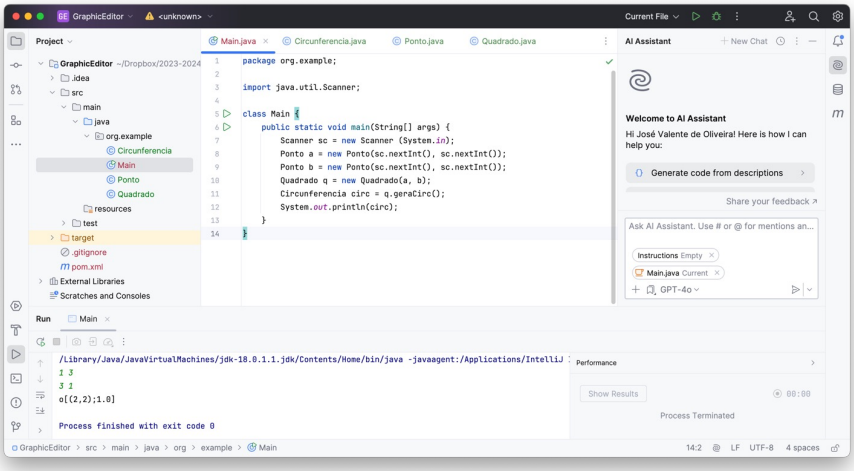
8

Documenting the code with an UML generator



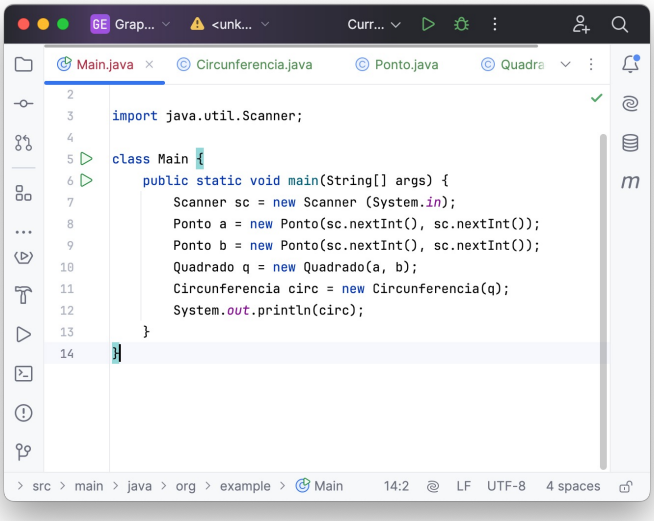
9

The initial client



10

What changes are needed to run this new client code?



11

Changes version 1.0

```
public class Circunferencia {
    private Ponto centro;
    private float raio;
//...
public Circunferencia (Quadrado q) {
    raio = q.getLado() / 2.0f;
    centro = q.getOrigem().soma (new Ponto((int) raio, (int) raio));
    }
}

public class Quadrado {
    private Ponto origem;
    private int lado;
//...
    public int getLado ()              { return this.lado; }
    public Ponto getOrigem ()          { return this.origem; }
}
```

```
Quadrado q = new Quadrado (P1, P2);
//Circunferencia c = q.geraCircunferencia (q);
Circunferencia c = new Circunferencia (q);
```

12

Changes version 2.0

```
public class Circunferencia {
    private Ponto centro;
    private float raio;
//...
public Circunferencia (Quadrado q) {
    Circunferencia c = q. geraCircunferencia ();
    centro = c.centro;
    raio    = c.raio;
    }
}
```

```
Quadrado q = new Quadrado (P1, P2);
//Circunferencia c = q.geraCircunferencia (q);
Circunferencia c = new Circunferencia (q);
```

13

Check point: what's wrong?

```
class Segmento {
   private final Ponto p1;
   private final Ponto p2;
   public Segmento(int x1, int y1, int x2, int y2) {
      this.p1 = new Ponto(x1, y1);
      this.p2 = new Ponto(x2, y2);
      if (x1 == x2 && y1 == y2) {
         System.out.println("Segmento:vi");
         System.exit(0);
      }
     //…
```

Submissão ao Problema E do Mooshak

6-14

14

Check point: Possible solution

```
class Segmento {
   private final Ponto p1;
   private final Ponto p2;
   /*  public Segmento(int x1, int y1, int x2, int y2) {
      this.p1 = new Ponto(x1, y1);
      this.p2 = new Ponto(x2, y2);
      if (x1 == x2 && y1 == y2) {
         System.out.println("Segmento:vi");
         System.exit(0);
    } */
    public Segmento(Ponto a, Ponto b) {
     if (a.equals(b)) {
         System.out.println("Segmento:vi");
         System.exit(0);
      }
      this.p1 = new Ponto(a.x(), a.y());
      this.p2 = new Ponto(b.x(), b.y());
     //…
```

6-15

15

Check point: what's wrong?

```
//…
 private double distanciaPontoParaSegmento(int cx, int cy, int x1, int y1, int x2, int y2) {
       double dx = x2 - x1;
       double dy = y2 - y1;

       double t = ((cx - x1) * dx + (cy - y1) * dy) / (dx * dx + dy * dy);
       double px, py;
       // Ponto mais próximo no segmento
       if (t < 0) { px = x1; py = y1;}
       else if (t > 1) { px = x2; py = y2; }
       else { px = x1 + t * dx; py = y1 + t * dy; }

       // Distância entre o centro do círculo e o ponto mais próximo
       return Math.sqrt(Math.pow(cx - px, 2) + Math.pow(cy - py, 2));
   }
}
```

Submissão ao Problema E do Mooshak

6-16

16

Check point: possible solution

```
//…
private double distanciaPara(Ponto c) {
        Ponto p = pontoMaisProximo (c);
        return c.dist(p);
    }
}
```